

JAVA SDE-2 INTERVIEW PREPARATION SERIES

FRAMEWORKS, DATA, AND MESSAGING - BOOK 01 OF 12 - STUDY STEP FW-01

MySQL for Java Backend Interviews

Relational Modeling, SQL, Indexes, Transactions, and Production Reasoning

BY

Vinay Reddy Kalluri

A prerequisite-first, SQL-first guide from relational contracts to InnoDB internals, indexes, transactions, JDBC, operations, and live SDE-2 interviews.

MYSQL | SQL | INDEXES | TRANSACTIONS | QUERY PLANS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to [Book 01 in the Frameworks, Data, and Messaging segment](#). If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Establish relational and SQL fundamentals before ORM abstractions, then build the InnoDB, indexing, concurrency, JDBC, migration, and diagnostic judgment expected from a Java SDE-2 engineer.

Readiness map

Decision	Evidence
START HERE	Data has relationships and invariants; A query or index determines latency; Correctness depends on transaction isolation.
PREPARE FIRST	JAVA 01 - Java Foundations or equivalent syntax fluency; Basic Java classes and collections.
MOVE ON WHEN	Model relational invariants and write correct joins, grouping, CTE, and window queries; Explain InnoDB B+ trees, clustered and secondary indexes, MVCC, redo/undo, locks, and deadlocks; Use EXPLAIN evidence, safe JDBC contracts, keyset pagination, migrations, recovery, and production diagnostics.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Frameworks Segment Position

START OF SEGMENT	YOU ARE HERE FW 01 - MySQL	FW 02 - Hibernate and JPA
------------------	-------------------------------	---------------------------

A note from Vinay

SQL skill is the ability to predict rows, locks, and work—not merely produce a query. Trace the logical result first, then use the plan and transaction evidence to explain production behavior.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - MySQL for Java Backend Interviews	4
Chapter 2 - Relational Modeling, Keys, Constraints, and Normalization	7
Chapter 3 - MySQL Types, NULL, Collation, Temporal Values, and Schema Safety	11
Chapter 4 - SQL Reasoning: Reads, Writes, Joins, Grouping, CTEs, and Windows	14
Chapter 5 - InnoDB Internals: B+trees, Buffer Pool, Redo, Undo, and MVCC	18
Chapter 6 - Index Design, EXPLAIN, and Query Performance	21
Chapter 7 - Transactions, Isolation, Locks, Deadlocks, and Retries	25
Chapter 8 - JDBC, Connection Pools, Batching, Timeouts, and Pagination	30
Chapter 9 - Migrations, Replication, Recovery, Observability, and Security	34
Chapter 10 - MySQL Live Interviews, Answered Questions, Practice, and Sources	37
Chapter 11 - Java 21 MySQL Interview Reasoning Companion	41
Part II - Practice and Handoff	44

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - MySQL for Java Backend Interviews

WHY IT WORKS | Learning Path and Relational First Principles

MySQL is not merely the place where a Java service stores objects. It is a concurrent system that enforces constraints, chooses physical access paths, writes recovery records, and coordinates transactions that may disagree about the same data.

From Vinay: In interviews, start with the business invariant and the SQL that preserves it. Add Hibernate or Spring Data only after you can explain what the database must do. That order keeps convenient APIs from hiding correctness.

What you will be able to do

By the end of this book, you should be able to:

- turn requirements into tables, keys, constraints, and query shapes;
- write SQL using joins, grouping, CTEs, and window functions;
- explain an InnoDB lookup from B+tree root to clustered row;
- choose an index from evidence instead of folklore;
- reason about MVCC, isolation, locks, deadlocks, and retries;
- build a safe JDBC boundary with pooling, batching, timeouts, and pagination; and
- discuss migrations, replication, recovery, security, and observability at SDE-2 depth.

The dependency order

TEXT

```
requirement
-> invariant and relational model
-> types, NULL rules, keys, and constraints
-> SQL result correctness
-> physical access path and EXPLAIN evidence
-> transaction and concurrency behavior
-> Java/JDBC resource boundary
-> production change, recovery, and operations
```

Do not jump to index tuning before proving that the query returns the right rows. Do not discuss transaction retries before identifying which statements are safe to repeat. Do not call a replica a backup before defining the recovery point you can restore.

One running domain

Most examples use a small ordering domain:

SQL

```
CREATE TABLE customer (  
    customer_id BIGINT PRIMARY KEY,  
    email VARCHAR(320) NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    created_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),  
    CONSTRAINT uq_customer_email UNIQUE (email),  
    CONSTRAINT chk_customer_status CHECK (status IN ('ACTIVE', 'BLOCKED'))  
);  
  
CREATE TABLE purchase_order (  
    order_id BIGINT PRIMARY KEY,  
    customer_id BIGINT NOT NULL,  
    request_key VARCHAR(64) NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    total_cents BIGINT NOT NULL,  
    version BIGINT NOT NULL DEFAULT 0,  
    created_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),  
    CONSTRAINT uq_order_request UNIQUE (customer_id, request_key),  
    CONSTRAINT fk_order_customer FOREIGN KEY (customer_id)  
        REFERENCES customer(customer_id),  
    CONSTRAINT chk_order_total CHECK (total_cents >= 0)  
);
```

The unique request key is not cosmetic. It turns "create this order once" into a database-enforced invariant even when a caller retries after a timeout.

The SDE-2 answer frame

For every database scenario, answer in this order:

- 1 **Correctness:** What invariant must always hold?
- 2 **Query:** Which rows and columns must be read or changed?
- 3 **Access path:** Which index can serve that query, and what evidence proves it?
- 4 **Concurrency:** What if two transactions do it at once?
- 5 **Failure:** What if the connection fails before, during, or after commit?
- 6 **Operations:** How will we migrate, observe, recover, and roll back?

First failure matrix

Failure	What the caller knows	Safe response
Validation fails before SQL	Nothing was written	Correct input; do not retry blindly
Constraint rejects statement	Transaction may be usable or marked for rollback by framework policy	Interpret the named invariant; roll back the unit of work
Deadlock victim	Current transaction was rolled back	Retry the whole idempotent transaction with a bound and jitter
Socket timeout during commit	Outcome may be unknown	Reconcile through idempotency key or read-after-reconnect
Replica is behind	Read may omit a committed write	Route consistency-sensitive read to source or use a causal strategy

Quick check

- 1 Why should a schema model an invariant even if Java validates it?
- 2 Why is "the API timed out" not proof that a transaction rolled back?
- 3 Which part of the answer frame comes before index design?

Practice

- **Foundation:** Write three invariants for an order-creation endpoint.
- **Interview Core:** Decide which invariant belongs in Java, which in SQL, and which in both.
- **SDE-2 Follow-up:** Describe how you would reconcile an unknown commit result without creating a duplicate order.

TRY IT | Readiness checkpoint

Continue when you can explain why a unique constraint is a concurrency tool, not just a validation convenience.

SDE-2 CORE CHAPTER

Chapter 2 - Relational Modeling, Keys, Constraints, and Normalization

Begin with facts, not Java classes

A relation represents facts with a stable meaning. A row should have an identity, columns should describe that fact, and constraints should reject states the business says are impossible.

For "a customer can place many orders," the foreign key belongs on the many side:

TEXT

```
customer 1 ----- * purchase_order
customer_id PK      order_id PK
                   customer_id FK
```

Avoid storing comma-separated order IDs in `customer`. That destroys referential integrity, makes joins awkward, and turns simple updates into string surgery.

Keys have different jobs

- A **primary key** is the table's chosen row identity. In InnoDB it also determines clustered organization.
- A **candidate key** is any minimal set of columns that uniquely identifies a row.
- A **natural key** has business meaning, such as a country code. It can change.
- A **surrogate key** is generated for identity, often `BIGINT` or a UUID-like value.
- A **foreign key** requires a referenced parent key and protects referential integrity.
- A **composite key** uses multiple columns; column order later matters for indexes.

An SDE-2 answer avoids "always use UUID" or "always use auto-increment." Compare insertion locality, global generation, exposure risk, storage width, secondary-index cost, and migration needs.

Constraints make races boring

Suppose two requests both run this check:

SQL

```
SELECT COUNT(*) FROM customer WHERE email = 'reader@example.com';
```

Both can see zero and both can insert. The application check improves the error message but does not close the race. This does:

SQL

```
ALTER TABLE customer
  ADD CONSTRAINT uq_customer_email UNIQUE (email);
```

Useful constraints include **NOT NULL**, **UNIQUE**, **PRIMARY KEY**, **FOREIGN KEY**, **CHECK**, and safe defaults. A default is not a substitute for a required business decision; it is appropriate only when omission has a stable meaning.

Normalization by dependency

Use normalization to stop one fact from being repeated in places that can disagree.

Consider:

TEXT

```
order_id | customer_id | customer_email | product_id | product_name | quantity
```

The order ID determines the customer, product ID determines product name, and an order can contain many products. Keeping everything in one table causes:

- **update anomaly:** changing a product name requires many rows;
- **insert anomaly:** a product cannot exist until an order does;
- **delete anomaly:** deleting the last order may erase the product fact.

A clearer model separates **customer**, **purchase_order**, **product**, and **order_item**. The order item can use (**order_id**, **product_id**) as a uniqueness constraint even if it has a surrogate row ID.

When denormalization is deliberate

Copying **unit_price_cents** into an order item is often correct: it records the price agreed at purchase time, not a duplicate of the product's current price. Denormalization is defensible when you state:

- 1 which read or historical requirement it serves;
- 2 which value is authoritative;
- 3 how the copy is updated or why it is immutable;
- 4 how drift is detected.

WATCH OUT | Relationship edge cases

Requirement	Schema decision	Common mistake
Delete customer but retain orders	soft-delete/status customer, or retain legal snapshot	<code>ON DELETE CASCADE</code> erases records
Exactly one active cart per customer	generated/explicit active key strategy plus unique constraint	check-then-insert race
Order lines cannot repeat product	<code>UNIQUE(order_id, product_id)</code>	enforce only in a Java <code>Set</code>
Money	integer minor units or intentional <code>DECIMAL(p, s)</code>	<code>FLOAT/DOUBLE</code> for exact currency
Optional relationship	nullable FK only when "unknown/absent" is valid	magic parent ID such as <code>0</code>

Predict the outcome

SQL

```
INSERT INTO customer(customer_id, email, status)
VALUES (1, 'a@example.com', 'ACTIVE');

INSERT INTO customer(customer_id, email, status)
VALUES (2, 'a@example.com', 'ACTIVE');
```

The second statement fails on `uq_customer_email`. The important result is not the vendor error text; it is that the database serialized the competing claims to one unique value.

Debug the design

Broken: `order(status VARCHAR(255), customer_email VARCHAR(255), product_ids TEXT)`.

Repair: introduce constrained status vocabulary, reference `customer_id`, and model order items as rows. Then name the delete rule and the uniqueness rule.

Interview angle

Question: "Would you keep a foreign key in a high-throughput service?"

Strong answer: Begin with the correctness it provides. Then discuss write cost, lock interactions, operational ownership, cross-database boundaries, and whether an event-driven repair process can truly replace the guarantee. Do not remove it because of a blanket performance claim.

Quick check

- 1 How is a candidate key different from the chosen primary key?
- 2 Why does a Java existence check not enforce uniqueness?
- 3 When can a copied column be a historical fact rather than harmful duplication?
- 4 What must you decide before choosing `ON DELETE CASCADE`?

Practice

- **Foundation:** Model students, courses, and enrollments with keys and constraints.
- **Interview Core:** Add idempotent payment-request creation without check-then-insert.
- **SDE-2 Follow-up:** Design a zero-downtime change from nullable to required `customer_id`.

Solution sketch

Use an enrollment bridge with a unique (`student_id`, `course_id`) pair. Give payment requests a business-scoped idempotency key. For the nullability change: add/accept the column, dual-write or backfill in bounded batches, verify, add the constraint using the safest supported deployment procedure, switch reads, and remove obsolete compatibility code later.

SDE-2 CORE CHAPTER

Chapter 3 - MySQL Types, NULL, Collation, Temporal Values, and Schema Safety

Types encode a domain and determine comparison, storage, conversion, index width, and client behavior. "Use `VARCHAR` for everything" throws away those guarantees.

Numeric choices

Need	Candidate	Reasoning
row count or identifier	sized integer, often <code>BIGINT</code>	exact arithmetic and predictable range
money	integer minor units or <code>DECIMAL(p, s)</code>	exact decimal value
scientific approximation	<code>DOUBLE</code>	range and approximate arithmetic
boolean flag	<code>BOOLEAN</code> alias/ <code>TINYINT</code> with domain convention	document allowed values

`DECIMAL(12, 2)` supports ten integer digits and two fractional digits. It does not automatically establish currency, rounding mode, or whether negative values are legal-those remain domain rules.

Text, bytes, character sets, and collations

`VARCHAR` stores text using a character set. `VARBINARY` stores bytes. A collation decides comparison and ordering behavior, including case and accent sensitivity. Therefore a unique index on an email or username can behave differently under different collations.

Do not silently lowercase values as a universal fix. Decide whether the identifier is case-sensitive, how Unicode normalization is handled, and whether display spelling is distinct from comparison spelling.

SQL

```
CREATE TABLE tag (  
  tag_id BIGINT PRIMARY KEY,  
  name VARCHAR(100) CHARACTER SET utf8mb4  
    COLLATE utf8mb4_0900_ai_ci NOT NULL,  
  CONSTRAINT uq_tag_name UNIQUE (name)  
);
```

This example is MySQL 8-specific. The collation is accent- and case-insensitive; use a binary or other deliberate collation when that is not the product rule.

NULL means unknown or absent-not zero or empty

SQL uses three-valued logic: a predicate can be **TRUE**, **FALSE**, or **UNKNOWN**.

SQL

```
SELECT * FROM customer WHERE status = NULL;    -- never the intended test
SELECT * FROM customer WHERE status IS NULL;    -- correct null test
```

NULL = **NULL** is **UNKNOWN**, not **TRUE**. In a **WHERE** clause, only **TRUE** rows survive.

A small truth table

Expression	Result
1 = 1	TRUE
1 = NULL	UNKNOWN
NULL = NULL	UNKNOWN
NULL IS NULL	TRUE
TRUE AND UNKNOWN	UNKNOWN
FALSE AND UNKNOWN	FALSE

NOT IN is a famous trap when the subquery can contain **NULL**:

SQL

```
-- Fragile when banned_customer.customer_id can be NULL
WHERE customer_id NOT IN (SELECT customer_id FROM banned_customer)

-- State the anti-join explicitly
WHERE NOT EXISTS (
  SELECT 1
  FROM banned_customer b
  WHERE b.customer_id = c.customer_id
)
```

Time is a data-model decision

Store an instant when the requirement is "a moment on the global timeline." Store a local date/time plus zone/recurrence rules when the requirement is "9:00 every day in Atlanta." A UTC timestamp alone cannot preserve a future civil schedule when zone rules change.

MySQL **TIMESTAMP** and **DATETIME** differ in range and timezone conversion behavior. Verify the selected MySQL version and JDBC driver behavior; do not rely on the server/session default timezone. In Java, prefer **Instant** for instants, **LocalDate** for dates, and **ZonedDateTime**/zone identifiers for civil schedules.

JSON is not a schema escape hatch

MySQL JSON is valuable for genuinely variable attributes, but frequently filtered or constrained fields deserve explicit modeling or generated columns with indexes. Ask:

- Can the database validate the shape?
- Which paths appear in predicates?
- How will the field evolve?
- Does partial update introduce lost-update risk?

Conversion and strictness failures

Implicit conversions can change results and index use. Compare compatible types in predicates. Run production with deliberate SQL modes and test migrations under the same modes. A development database that truncates invalid values while production rejects them creates false confidence.

Edge-case matrix

Case	Risk	Safer decision
<code>VARCHAR</code> number compared to integer	conversion and surprising matches	store and bind the correct numeric type
nullable unique value	multiple <code>NULL</code> rows may be permitted	model "one absent/active" rule explicitly
<code>DOUBLE</code> price	equality and rounding surprises	exact representation
default timezone differs	shifted instants	explicit UTC/session/driver contract
case-insensitive collation	unexpected duplicate rejection	match collation to product identity rule
oversized indexed text	larger B+tree and fewer entries per page	size from domain and query evidence

Quick check and practice

- 1 Why does `column = NULL` not find nulls?
 - 2 Why can `NOT IN` with a nullable subquery return no rows?
 - 3 What product decision precedes choosing a collation?
 - 4 Distinguish an instant from a recurring local schedule.
- **Foundation:** Rewrite five magic-value columns as nullable or constrained domains.
 - **Interview Core:** Choose types for amount, currency, placed instant, and delivery-local window.
 - **SDE-2 Follow-up:** Plan a collation change for a unique username without accepting or losing duplicates.

SDE-2 CORE CHAPTER

Chapter 4 - SQL Reasoning: Reads, Writes, Joins, Grouping, CTEs, and Windows

The best SQL answers begin by defining the output grain: **what does one result row represent?** Most duplicate-row bugs begin when a candidate never answers that question.

Reads and writes with explicit intent

Avoid `SELECT *` at service boundaries. Request the columns the contract needs; that reduces coupling, transfer, and accidental exposure.

SQL

```
SELECT order_id, status, total_cents, created_at
FROM purchase_order
WHERE customer_id = ?
ORDER BY created_at DESC, order_id DESC
LIMIT ?;
```

The second sort key makes ordering deterministic when timestamps tie.

For conditional updates, put the precondition in the statement:

SQL

```
UPDATE purchase_order
SET status = 'PAID', version = version + 1
WHERE order_id = ?
  AND status = 'CREATED'
  AND version = ?;
```

An affected-row count of zero is information: missing row, wrong state, or concurrent version change. The service must distinguish them only if its API contract requires that distinction.

Joins: preserve the intended side

SQL

```
SELECT c.customer_id, COUNT(o.order_id) AS order_count
FROM customer c
LEFT JOIN purchase_order o
  ON o.customer_id = c.customer_id
  AND o.status = 'PAID'
GROUP BY c.customer_id;
```

Putting the order status in `ON` retains customers with zero paid orders. Moving it to `WHERE` removes null-extended rows and effectively turns the query into an inner join.

Join multiplication

Joining orders to both items and payments can multiply rows (`items × payments`) before aggregation. Repair it by aggregating each many-side to the required grain first, then joining the summaries.

Grouping and HAVING

`WHERE` filters input rows before grouping. `HAVING` filters groups after aggregation.

SQL

```
SELECT customer_id, SUM(total_cents) AS paid_total
FROM purchase_order
WHERE status = 'PAID'
GROUP BY customer_id
HAVING SUM(total_cents) >= 100_000;
```

Keep selected nonaggregated columns consistent with the grouping contract. Do not depend on permissive modes returning an arbitrary value.

Subqueries and CTEs

A CTE names an intermediate relation; it does not guarantee materialization or performance improvement.

SQL

```
WITH customer_totals AS (
  SELECT customer_id, SUM(total_cents) AS total_cents
  FROM purchase_order
  WHERE status = 'PAID'
  GROUP BY customer_id
)
SELECT c.customer_id, t.total_cents
FROM customer c
JOIN customer_totals t ON t.customer_id = c.customer_id
WHERE t.total_cents >= 100_000;
```

Recursive CTEs can traverse bounded hierarchies, but graph cycle handling and recursion depth must be explicit.

Window functions keep row detail

GROUP BY collapses rows. A window function computes across a partition while retaining each row.

SQL

```
SELECT order_id,
       customer_id,
       total_cents,
       ROW_NUMBER() OVER (
         PARTITION BY customer_id
         ORDER BY created_at DESC, order_id DESC
       ) AS recency_rank,
       SUM(total_cents) OVER (
         PARTITION BY customer_id
         ORDER BY created_at, order_id
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM purchase_order;
```

For running totals, write the window frame deliberately. Peer rows can make default **RANGE** behavior differ from the row-by-row result you imagined.

Top N per group

SQL

```
WITH ranked AS (
  SELECT o.*,
         ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY created_at DESC, order_id DESC
         ) AS rn
  FROM purchase_order o
)
SELECT order_id, customer_id, total_cents, created_at
FROM ranked
WHERE rn <= 3;
```

EXISTS, IN, and counting

Use **EXISTS** when the question is whether a matching row exists; it communicates that no count is needed. Optimizers may transform equivalent forms, so verify the plan rather than asserting one syntax is universally faster.

Predict and debug

Question: Why can this count be too large?

SQL

```
SELECT o.customer_id, COUNT(*)
FROM purchase_order o
JOIN order_item i ON i.order_id = o.order_id
JOIN payment p ON p.order_id = o.order_id
GROUP BY o.customer_id;
```

Answer: each order appears once per item-payment combination. Decide whether the metric counts orders, items, or payments, aggregate at that grain, and only then join.

Quick check

- 1 What does one result row represent in a top-three-orders query?
- 2 When does a `LEFT JOIN` become effectively inner?
- 3 Why is a CTE not automatically faster?
- 4 What is retained by a window function that grouping removes?

Practice

- **Foundation:** Return customers with zero orders.
- **Interview Core:** Return the second-highest distinct order total per customer.
- **Interview Core:** Compute a seven-row moving average with deterministic order.
- **SDE-2 Follow-up:** Diagnose a revenue query that doubled after a payments join.

Solution direction

Use `LEFT JOIN ... IS NULL` or `NOT EXISTS` for zero orders; use `DENSE_RANK` for second-highest distinct value; define a `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` frame; and aggregate each many-side at order grain before combining.

SDE-2 CORE CHAPTER

Chapter 5 - InnoDB Internals: B+trees, Buffer Pool, Redo, Undo, and MVCC

This chapter gives enough implementation intuition to explain performance and correctness. It does not ask you to memorize page-file internals that vary by version.

The row lookup path

InnoDB organizes the primary-key index as a clustered B+tree. Leaf records contain the row data. A secondary-index leaf contains the secondary key plus the primary-key value, so a noncovering secondary lookup often needs a second B+tree traversal.

TEXT

```
SELECT total_cents
FROM purchase_order
WHERE customer_id = 42;

secondary B+tree (customer_id, primary key)
    root -> branch -> leaf: (42, order_id=9001)
                        |
                        v
clustered primary-key B+tree
    root -> branch -> leaf: full order row
```

If the secondary index also contains the projected column, the plan may be covering and avoid the clustered lookup:

SQL

```
CREATE INDEX ix_order_customer_total
ON purchase_order(customer_id, total_cents);
```

Coverage is query-specific and not free: wider indexes consume memory, add write amplification, and reduce entries per page.

Why primary-key shape matters

Every secondary entry carries the primary key. A wide or random primary key can enlarge all secondary indexes and reduce insertion locality. That does not make random identifiers "wrong"; it makes the trade-off measurable.

InnoDB creates a hidden clustered key when no suitable primary/unique key exists. Relying on it gives the application no stable explicit identity and is poor schema design.

The buffer pool

The buffer pool caches data and index pages. A logical read may be served from memory; a miss requires storage I/O. A benchmark immediately after restart and one after warm-up answer different questions.

Dirty pages contain committed or uncommitted changes not yet written to their data-file locations. Commit durability does **not** require every dirty data page to be flushed immediately.

Redo versus undo

TEXT

```
UPDATE row
|-- undo information: previous version / rollback + MVCC history
|-- buffer-pool page becomes dirty
|-- redo record: enough physical/logical change information for recovery
`-- commit durability depends on log-flush configuration
```

- **Redo log** supports crash recovery by replaying durable changes whose data pages were not yet persisted.
- **Undo records** support rollback and older row versions used by consistent reads.
- **Binary log** is a server-level change stream used for replication and point-in-time recovery; it is not the same as InnoDB redo.

Do not say "undo always contains a full copy" or "commit writes the table row to disk." Those oversimplifications produce incorrect incident reasoning.

MVCC visibility intuition

A consistent read uses a read view to decide which row version is visible. A transaction may therefore read an older committed version while a newer version exists.

TEXT

```
row v3 (created by active T9) -> undo points to v2 (committed before read view)
                               ^ visible version for this read
```

Long-running transactions can keep old versions relevant and delay purge, increasing history and storage pressure. "Readers do not block writers" is only a partial statement: locking reads, writes, metadata operations, and resource pressure still interact.

Page split and locality intuition

When insertion targets a full B+tree page, the tree may split or reorganize. Sequential keys usually append with better locality; random keys distribute writes but may cause more scattered page work. The real decision also includes hot-spotting, distributed ID generation, privacy, and secondary-index width.

Internal-to-symptom map

Internal behavior	Observable symptom	Evidence to collect
poor buffer-pool locality	storage reads and latency after cache misses	buffer-pool metrics, I/O, plan, working set
noncovering range lookup	many clustered lookups	EXPLAIN ANALYZE , rows examined, projection
long read view	growing undo/history and purge lag	transaction age, history metrics
random wide primary key	larger indexes and write cost	index sizes, page behavior, write workload
checkpoint/log pressure	periodic write stalls	redo/checkpoint and fsync latency metrics

Interview question

"Does an index store pointers to table rows?"

Answer precisely for InnoDB: the clustered primary-key leaf is the row. A secondary leaf stores its key and the primary-key value; the engine uses that key for a clustered lookup unless the secondary index covers the query. Avoid applying heap-table terminology to every engine.

Quick check and practice

- 1 Why can a secondary lookup require two B+tree traversals?
 - 2 How can commit be durable before a data page reaches its final file location?
 - 3 What does undo provide besides rollback?
 - 4 Why can a long-running reader create operational pressure?
- **Foundation:** Draw the lookup for `WHERE email = ?` with a unique secondary index.
 - **Interview Core:** Explain why adding projection columns can help and hurt.
 - **SDE-2 Follow-up:** Given a latency regression only after deployment restart, distinguish cold-cache behavior from a bad plan.

SDE-2 CORE CHAPTER

Chapter 6 - Index Design, EXPLAIN, and Query Performance

An index is an ordered data structure maintained on every relevant write. Design it from a real query shape-not from a list of frequently mentioned columns.

The query-shape worksheet

For each important query, record:

TEXT

FROM/JOIN tables:
equality predicates:
range predicates:
required order:
projected columns:
expected matches and skew:
latency/throughput target:
write rate and index budget:

Example:

SQL

```
SELECT order_id, total_cents, created_at
FROM purchase_order
WHERE customer_id = ?
      AND status = 'PAID'
      AND created_at < ?
ORDER BY created_at DESC, order_id DESC
LIMIT 50;
```

A candidate is:

SQL

```
CREATE INDEX ix_order_customer_status_created_id
ON purchase_order(customer_id, status, created_at DESC, order_id DESC);
```

The equality columns lead, followed by the range/order columns. This is a useful heuristic, not a law. Low-cardinality status can still be valuable after customer because the combined prefix is selective and matches the query. Measure alternate orders under real data distribution.

Leftmost-prefix behavior

For (`customer_id`, `status`, `created_at`), the ordering can efficiently support predicates beginning with `customer_id`, and often with both `customer_id` and `status`. A predicate only on `status` cannot generally seek through the first dimension of this B+tree.

After a range on one key part, later parts may still help filtering or coverage but frequently cannot narrow the same contiguous search interval. Read the actual plan for your version and query.

Sargability

Keep the indexed column available for direct comparison.

SQL

```
-- Often prevents a simple range seek on created_at
WHERE DATE(created_at) = '2026-08-02'

-- Half-open range preserves all times and index ordering
WHERE created_at >= '2026-08-02 00:00:00'
  AND created_at < '2026-08-03 00:00:00'
```

Other common problems: mismatched types, leading-wildcard patterns, arithmetic on the indexed column, and collation conversions.

Reading EXPLAIN as evidence

Important signals include:

- chosen and candidate keys;
- access type and key parts used;
- estimated rows and filtering;
- additional work such as temporary tables or filesort;
- join order;
- actual timing and rows from `EXPLAIN ANALYZE` where safe.

Do not rank a plan by one field alone. "Using filesort" is not automatically bad, and "uses an index" can still mean scanning most of it.

Estimate-versus-actual reasoning

If the optimizer estimates 10 rows but observes 500,000, investigate stale/inadequate statistics, skew, correlated columns, parameter distribution, and predicate conversions. An index may be structurally correct yet rejected because the estimate says it is expensive.

Covering and redundant indexes

Coverage avoids clustered-row lookup, but adding every selected column creates a large write-heavy index. Review whether (a) , (a, b) , and (a, b, c) are all needed; longer indexes sometimes subsume shorter ones, but uniqueness, sort direction, size, and workload can justify both.

Pagination plans

Deep offset pagination performs and discards earlier work:

SQL

```
... ORDER BY created_at DESC, order_id DESC  
LIMIT 50 OFFSET 500000;
```

Keyset pagination continues after the last key:

SQL

```
WHERE (created_at < ?)  
      OR (created_at = ? AND order_id < ?)  
ORDER BY created_at DESC, order_id DESC  
LIMIT 50;
```

MySQL row constructors may express an equivalent tuple comparison where types/order align. The cursor must contain every tie-breaker and be opaque/tamper-resistant if exposed to clients.

Offset still fits small admin screens and random page-number navigation. Keyset fits stable forward/backward traversal at scale.

Performance investigation loop

- 1 Confirm user-visible symptom and time window.
- 2 Capture normalized query, bind-value distribution, call rate, and rows returned.
- 3 Inspect plan and actual work on representative data.
- 4 Check waits: CPU, storage, locks, connection pool, network.
- 5 Change one hypothesis-query, index, stats, data model, or workload.
- 6 Re-measure p50/p95/p99 and write-side cost.

WATCH OUT | Edge cases

Symptom	Plausible cause	Counterexample to avoid
index ignored	query returns large fraction	"optimizer is broken"
query slow only for one tenant	data skew	average cardinality assumption
plan changed after deploy	statistics/data/query shape changed	blaming application version only
fast query, slow endpoint	pool wait/network/serialization	adding an index
composite index not useful	missing leading prefix/type mismatch	adding each column separately

Interview drill

Question: "Index every foreign key and every **WHERE** column?"

Answer: Foreign-key lookup and parent-delete checks often benefit, but indexes must follow workload. Several single-column indexes do not automatically replace one ordered composite index. Each index has write, memory, storage, and maintenance cost.

Practice

- **Foundation:** Explain why `(last_name, first_name)` does not efficiently seek by first name alone.
- **Interview Core:** Design and defend an index for the sample cursor query.
- **SDE-2 Follow-up:** An index makes reads 40% faster but doubles insert p99. Define the decision evidence.

TRY IT | Readiness checkpoint

You are ready when you can say which rows an index narrows, which ordering it provides, whether it covers, and what it costs to maintain.

SDE-2 CORE CHAPTER

Chapter 7 - Transactions, Isolation, Locks, Deadlocks, and Retries

A transaction is a boundary for atomic database work. It is not a magic wrapper around network calls, message publication, or external APIs.

ACID in operational language

- **Atomicity:** the database commits the transaction's effects or rolls them back.
- **Consistency:** declared database rules plus correct transaction logic preserve invariants; the database cannot invent missing business rules.
- **Isolation:** concurrent executions observe behavior defined by the isolation level and statement types.
- **Durability:** after acknowledged commit, recovery should preserve effects according to configured durability guarantees.

Autocommit changes the unit of work

With autocommit, each statement is normally its own transaction. A read-then-write sequence is therefore not atomic unless you create an explicit transaction or encode the condition in one statement.

SQL

```
UPDATE inventory
SET available = available - 1
WHERE sku = ? AND available > 0;
```

If affected rows is one, stock was reserved atomically. If zero, it was missing or exhausted. This is often clearer than `SELECT`, Java decrement, `UPDATE`.

Isolation anomalies

Anomaly	Shape	Protection approach
dirty read	read another transaction's uncommitted data	use supported normal isolation; InnoDB does not expose uncommitted versions in ordinary reads as a general design tool
nonrepeatable read	same row reads differently	snapshot semantics or locking read, depending requirement
phantom	repeated predicate yields new matching rows	predicate/range locking or serializable strategy when invariant needs it
lost update	later writer overwrites earlier change	conditional update, version column, or lock
write skew	transactions update different rows after shared predicate	materialize/lock invariant or serialize decision

MySQL/InnoDB behavior depends on isolation level and whether a read is consistent (**SELECT**) or locking (**SELECT ... FOR UPDATE**). State both. Default isolation can also be configured; verify the environment instead of reciting a default as universal.

MVCC is not the same as "no locks"

Ordinary consistent reads use versions. Writes acquire locks. Locking reads acquire locks. InnoDB may use record, gap, or next-key locking to protect searched ranges depending on isolation, index, uniqueness, and predicate.

SQL

```
START TRANSACTION;
SELECT available
FROM inventory
WHERE sku = ?
FOR UPDATE;
-- validate and update quickly
COMMIT;
```

Use locking reads when the invariant truly needs serialization and keep the transaction small. Do not hold a row lock while calling a payment provider.

Deadlocks are cycles, not merely slow locks

TEXT

```
T1 holds order 10, waits for order 20
T2 holds order 20, waits for order 10
    -> cycle -> one victim is rolled back
```

Prevention and response:

- 1 access shared resources in a stable order;
- 2 use selective indexes so statements do not lock more range than intended;
- 3 keep transactions short and avoid user/network wait inside them;
- 4 log enough context to diagnose the cycle;
- 5 retry the **entire** transaction only when its side effects are safe and the error is classified retryable;
- 6 bound retries, add jitter, and surface persistent contention.

A lock timeout is waiting beyond a configured limit; a deadlock is a detected cycle. Both may invite a retry policy, but their evidence and remediation differ.

Unknown commit outcome

If the socket breaks around **COMMIT**, the client may not know whether the server committed. Retrying a non-idempotent insert can duplicate work.

Use a unique request key:

SQL

```
INSERT INTO payment_request(request_key, order_id, amount_cents, status)
VALUES (?, ?, ?, 'STARTED');
```

After reconnect, query by `request_key`. The same pattern coordinates an outbox record for later message publication without pretending SQL and Kafka share one atomic transaction.

Optimistic versus pessimistic control

Optimistic: update with expected version and retry/reject on zero rows. Best when collisions are uncommon and work can be repeated.

SQL

```
UPDATE purchase_order
SET status = ?, version = version + 1
WHERE order_id = ? AND version = ?;
```


Pessimistic: lock before decision. Best when collision is likely or late failure is very expensive, but it consumes locks/connections and can deadlock.

Concurrency case study: reserve two seats

Naive logic reads each seat, calls another service, then updates. It holds locks too long and can partially act externally.

A stronger design:

- 1 sort seat IDs;
- 2 transactionally lock/update both in the same order;
- 3 insert reservation and outbox event with unique request key;
- 4 commit;
- 5 publish asynchronously; consumer deduplicates;
- 6 compensate an expired reservation through an explicit state transition.

Failure matrix

Event	Transaction state	Retry?
syntax/constraint error	rollback unit of work	fix request/code, generally no
deadlock victim	rolled back	bounded whole-transaction retry
lock timeout	framework/driver may mark failure; rollback explicitly	only after classification and policy
connection lost before begin	no work known	reconnect; repeat if safe
connection lost during commit	unknown outcome	reconcile by idempotency key
Java caught exception but transaction continues	dangerous partial-work assumption	mark rollback or fail boundary

Practice

- **Foundation:** Explain why autocommit breaks a read-then-write unit.
- **Interview Core:** Protect coupon redemption limited to 100 uses.
- **Interview Core:** Draw a two-row deadlock and remove its cycle.
- **SDE-2 Follow-up:** Design retry handling when a database transaction also schedules an email.

Solution direction

Prefer one conditional increment guarded by `< 100`, or a locked counter when more validation is required. Use an outbox for email intent; do not send the email inside the retried transaction.

SDE-2 CORE CHAPTER

Chapter 8 - JDBC, Connection Pools, Batching, Timeouts, and Pagination

JDBC is where database guarantees meet Java resource management. A correct SQL statement can still fail operationally through pool exhaustion, transaction leakage, unsafe retries, or type conversion.

A safe transaction skeleton

JAVA

```
static boolean markPaid(
    DataSource dataSource,
    long orderId,
    long expectedVersion) throws SQLException {
    String sql = ""
        UPDATE purchase_order
        SET status = 'PAID', version = version + 1
        WHERE order_id = ? AND status = 'CREATED' AND version = ?
        "";

    try (Connection connection = dataSource.getConnection()) {
        boolean originalAutoCommit = connection.getAutoCommit();
        try {
            connection.setAutoCommit(false);
            try (PreparedStatement statement = connection.prepareStatement(sql)) {
                statement.setLong(1, orderId);
                statement.setLong(2, expectedVersion);
                int changed = statement.executeUpdate();
                if (changed != 1) {
                    connection.rollback();
                    return false;
                }
            }
            connection.commit();
            return true;
        } catch (SQLException failure) {
            connection.rollback();
            throw failure;
        } finally {
            connection.setAutoCommit(originalAutoCommit);
        }
    }
}
```

Production pools usually reset connection state on return, but application code should not depend on leaking altered autocommit, isolation, read-only flags, or session variables. Framework transaction managers centralize this boundary; understand the equivalent behavior before using them.

Prepared statements

Prepared statements separate SQL structure from data, prevent ordinary parameter injection, and provide typed binding. They do not parameterize table names, column names, or arbitrary sort expressions; allow-list those structural choices.

Use `setNull(index, sqlType)` when needed. Check JDBC driver behavior for temporal types and unsigned/narrowing conversions. Avoid constructing `IN ( . . . )` by concatenating untrusted text; generate placeholders or use a supported collection strategy.

Pool sizing is a queueing decision

A connection is a scarce concurrent resource. A larger pool can increase database contention rather than throughput.

Estimate from measured service time and arrival rate, database connection budget, number of application instances, and acceptable wait. Monitor separately:

- pool acquisition wait;
- active/idle/pending connections;
- query execution time;
- transaction duration;
- timeout/cancel result.

Set acquisition, connect, socket, statement, and transaction deadlines deliberately. A timeout exception does not always prove server-side work stopped; cancellation and outcome reconciliation are part of the contract.

Batch writes

JAVA

```
try (PreparedStatement statement = connection.prepareStatement(
    "INSERT INTO order_item(order_id, product_id, quantity) VALUES (?, ?, ?)") {
    for (OrderItem item : items) {
        statement.setLong(1, item.orderId());
        statement.setLong(2, item.productId());
        statement.setInt(3, item.quantity());
        statement.addBatch();
    }
    int[] counts = statement.executeBatch();
}
```

Batching reduces round trips, but huge transactions expand locks, logs, memory, and failure replay. Chunk by measured size. Driver rewrite settings may affect whether a JDBC batch becomes a multi-value statement. Generated keys and partial batch failures require explicit tests.

Streaming and fetch behavior

Reading a million rows into a `List` is not streaming. Configure fetch behavior supported by the driver/version, process incrementally, hold the connection only as long as necessary, and define transaction consistency. A streaming result ties up a connection; backpressure must reach the caller.

Cursor pagination in Java

JAVA

```
record OrderCursor(Instant createdAt, long orderId) {}

String sql = """
    SELECT order_id, total_cents, created_at
    FROM purchase_order
    WHERE customer_id = ?
      AND (created_at < ? OR (created_at = ? AND order_id < ?))
    ORDER BY created_at DESC, order_id DESC
    LIMIT ?
    """;
```

Fetch `pageSize + 1` to determine whether another page exists, return only `pageSize`, and encode the last row's complete ordering key. Decide how inserts/deletes between requests affect the product experience.

Resource and failure matrix

Mistake	Result	Repair
no try-with-resources	pool leak	close result, statement, connection deterministically
catch without rollback	ambiguous transaction state	rollback then propagate/translate
retry one statement	broken unit-of-work invariant	retry whole idempotent transaction
log bind values indiscriminately	secrets/PII exposure	structured redacted diagnostics
pool much larger than DB capacity	queue moves into database	budget globally and load test
deep <code>OFFSET</code>	increasing discarded work	keyset when navigation permits

Quick check and practice

- Which connection settings can leak across pool borrowers?
 - Why is a query timeout not proof of rollback?
 - What makes a batch too large?
 - Why must a cursor contain a tie-breaker?
- Foundation:** Implement a query using try-with-resources and typed mapping.

- **Interview Core:** Add bounded deadlock retry around the whole transaction.
- **SDE-2 Follow-up:** Diagnose an endpoint whose SQL is 20 ms but pool acquisition is 900 ms.

SDE-2 CORE CHAPTER

Chapter 9 - Migrations, Replication, Recovery, Observability, and Security

Production database work is a lifecycle: safely change a live schema, route traffic, detect trouble, and recover data. A migration that is syntactically valid can still block writes or break old application instances.

Expand, migrate, contract

For a risky rename or type change:

- 1 **Expand:** add the new compatible column/table/index.
- 2 **Deploy compatibility:** write both forms when necessary; read with a safe fallback.
- 3 **Backfill:** bounded, restartable batches with checkpoints and rate limits.
- 4 **Verify:** counts, nulls, checksums/domain invariants, and query plans.
- 5 **Switch:** move reads to the new representation.
- 6 **Contract later:** stop old writes, observe, then remove obsolete schema.

DDL algorithms, lock modes, and online behavior vary by MySQL version and operation. Test on production-like size; "online" can still consume resources or wait on metadata locks.

Index rollout

Before adding an index, capture the query and baseline. During rollout monitor DDL progress, replication lag, I/O, and write latency. After rollout confirm the plan and business latency. Have a rollback/removal criterion-but remember removing a large index is also an operation.

Replication is asynchronous unless you prove otherwise

MySQL replication uses the binary log to transmit committed changes. A replica may lag. Therefore read-after-write from a replica can return older data.

Choose per endpoint:

- source read for immediate consistency;
- session/causal marker with wait/routing logic;
- tolerate stale results and communicate the product behavior;
- cache/replica only for data whose staleness budget permits it.

Replication improves availability/read capacity; it also reproduces accidental deletes. **A replica is not a backup.**

Recovery objectives

- **RPO:** maximum acceptable data loss measured in time or transactions.
- **RTO:** maximum acceptable time to restore service.

Meet them with a tested combination of full/incremental backups, binary logs for point-in-time recovery, retention, encryption, separate failure domains, and documented restore procedures. A backup is not proven until a restore test validates the application's data invariants.

Failover correctness

Promotion can expose lost/unapplied writes depending on topology and acknowledgement policy. Clients need connection discovery/routing, retry discipline, and idempotency. Fencing prevents an old source from accepting writes after a new one is promoted.

What to observe

TEXT

```
application: request rate, errors, p95/p99, pool wait, transaction duration
query: normalized digest, calls, total/mean/tail time, rows examined/returned
database: CPU, I/O, buffer pool, redo/checkpoint, locks/deadlocks, temp work
replication: lag/apply state/errors, source position
capacity: data/index size, connections, growth, backup duration
```

Slow-query evidence is a starting point, not a verdict. Correlate with trace/request identifiers without logging credentials or full sensitive bind values.

Security baseline

- separate application, migration, and administrative identities;
- least privilege and narrow network access;
- TLS in transit and managed encryption/keys at rest;
- parameterized SQL plus allow-listed structural input;
- secret rotation without source-control exposure;
- audited privileged operations;
- data classification, minimization, retention, and deletion verification;
- backups protected to the same or stronger standard as the source.

Encryption does not fix SQL injection; parameter binding does not fix excessive privileges; masking a log does not fix an overbroad database export. Use layered controls.

Operational failure table

Event	First concern	Immediate evidence	Durable fix
migration stalls writes	metadata/row locks	process/lock waits, DDL phase	safer rollout and rehearsal
replica returns missing order	lag/read routing	replay position, request route	causal/source read policy
backups succeed but restore fails	false recoverability	restore logs/invariant checks	automated restore drills
connection storm	retry amplification	pool pending, connection rate	backoff, budgets, admission control
credentials leaked	scope and dwell time	audit/connect logs	rotate, revoke, reduce privilege

Practice

- **Foundation:** Define RPO/RTO for a shopping cart and completed payments.
- **Interview Core:** Plan a non-null column addition with old and new app versions live.
- **Interview Core:** Explain why a read replica can violate read-after-write.
- **SDE-2 Follow-up:** Design an evidence-driven failover drill with rollback and success criteria.

SDE-2 CORE CHAPTER

Chapter 10 - MySQL Live Interviews, Answered Questions, Practice, and Sources

The questions below are written as interview dialogue. Read the prompt, pause, answer aloud, then compare with the reasoning. Strong answers state assumptions and failure behavior; they do not recite feature lists.

Live interview 1: duplicate payments

Interviewer: "A client times out and retries `POST /payments`. How do you prevent two charges?"

Candidate: "I would require a client-scoped idempotency key and persist it under a unique constraint with the payment intent. The transaction claims that key and writes an outbox event. If commit acknowledgement is lost, the retry reads the existing row instead of issuing a second charge. The payment provider also receives a stable idempotency key. I would define how failed and expired attempts reuse or reject that key."

Follow-up: Why not `SELECT` before `INSERT`?

Answer: Competing requests can both observe absence. The unique constraint arbitrates the race.

Live interview 2: index design

Interviewer: "Fetch the newest 50 paid orders for one customer. What index?"

Candidate: "First I need the exact predicate and deterministic order. For `customer_id = ?, status = 'PAID'`, ordered by `(created_at DESC, order_id DESC)`, I would test `(customer_id, status, created_at DESC, order_id DESC)`. It narrows equality first and serves the cursor order. I would compare row estimates and actual work under skew, and assess write/index-size cost. I would not add projected values for coverage until evidence justifies the width."

Live interview 3: overselling inventory

Interviewer: "Two buyers see one item. Stop overselling."

Candidate: "For a simple counter I prefer one atomic conditional update: decrement where available is positive, then require one affected row. If allocation spans rows or rules, I use a short explicit transaction, stable lock order, and `SELECT ... FOR UPDATE`/conditional writes. I keep payment calls outside the lock window and make reservation expiry a state transition."

Live interview 4: deadlocks after feature launch

Interviewer: "Deadlocks rose 20×. What do you do?"

Candidate: "Deadlocks are expected signals, not solved by increasing a timeout. I capture deadlock graphs plus query digests and business IDs, identify inconsistent resource order or widened scans, and inspect indexes. I make resource order deterministic, shorten the transaction, and add bounded classified retry for idempotent units. I verify retry amplification and tail latency after the change."

Live interview 5: replica lag

Interviewer: "The order confirmation page sometimes says 'not found'."

Candidate: "I would trace whether the write went to the source and the immediate read went to an asynchronous replica. For this flow I would use source/causal read-after-write or carry a consistency token and wait within a budget. I would measure lag and define fallback; making every read primary is simple but has capacity cost."

Live interview 6: zero-downtime schema change

Interviewer: "Rename a heavily used column."

Candidate: "I would avoid a one-step rename while mixed application versions run. Add the new column, deploy compatible dual-write/backfill logic, migrate in restartable batches, validate equality and query plans, switch reads, observe, then remove old compatibility later. I would rehearse DDL locks and replication impact on production-like data."

Live interview 7: intermittent slow query

Interviewer: "The same SQL is usually 10 ms but sometimes 4 seconds."

Candidate: "I separate pool wait from server time, preserve bind distribution, and correlate slow instances with lock waits, storage/cache misses, plan changes, temp work, and tenant skew. An average **EXPLAIN** without the slow bind is insufficient. I form one hypothesis and compare actual rows and wait evidence before changing an index."

Rapid answered questions

- 1 **Primary key versus unique key?** Both enforce uniqueness; one chosen primary key drives InnoDB clustering and cannot be null. Other candidate keys remain unique constraints.
- 2 **Does `COUNT(column)` count nulls?** No; it counts non-null values. `COUNT(*)` counts rows.
- 3 **WHERE versus HAVING?** `WHERE` filters input rows; `HAVING` filters grouped results.
- 4 **Why can `NOT IN` surprise?** A null in the compared set can make comparisons unknown. Use a null-safe design or correlated `NOT EXISTS`.
- 5 **Is a CTE materialized?** Not as a universal rule; the optimizer may merge or materialize it.
- 6 **Is an index always faster?** No. Low selectivity, random I/O, maintenance, and poor estimates can make a scan cheaper.
- 7 **Why include a tie-breaker in sorting?** Equal primary sort values otherwise produce unstable order and broken cursors.
- 8 **What is a covering index?** One containing all columns required for that query's predicates/order/projection, avoiding clustered lookup.
- 9 **MVCC means no locks?** No. Consistent reads use versions; writes and locking reads acquire locks.
- 10 **Deadlock versus lock timeout?** A deadlock is a cycle the engine breaks; a timeout is excessive waiting without necessarily a cycle.
- 11 **Can every deadlock be retried?** Only retry the whole transaction when operations are idempotent/reconcilable, with bounds and jitter.
- 12 **What is an unknown outcome?** The client lost acknowledgement near commit and cannot infer whether work committed.
- 13 **Why is a replica not a backup?** It can reproduce corruption/deletion and does not itself provide historical recovery points.
- 14 **Offset versus keyset?** Offset supports page numbers but deep pages discard work; keyset continues from ordered values but lacks arbitrary jumps.
- 15 **Why can a bigger pool hurt?** It moves queuing into MySQL and increases contention, memory, and context switching.
- 16 **Does commit flush every data page?** No; durability uses logging/configuration while dirty pages can be written later.
- 17 **Why not log all SQL values?** They can expose secrets/PII and generate cost; use redacted structured diagnostics.
- 18 **When use a foreign key?** When the database owns both sides and referential integrity matters; discuss measured cost and lifecycle, not folklore.

Cumulative assessment

Design an order service that supports idempotent creation, paid-order cursor listing, stock reservation, and read replicas. Deliver:

- 1 tables, keys, constraints, and data types;
- 2 the four most important SQL statements;
- 3 indexes justified by those statements;
- 4 concurrency timeline for two buyers;
- 5 JDBC transaction/resource boundary;
- 6 migration and backfill plan;
- 7 dashboards and recovery test;
- 8 answers for deadlock, timeout-at-commit, and replica-lag failures.

Assessment rubric

Level	Evidence
Foundation	valid model and SQL; explains nulls and joins
Interview Core	deliberate indexes, atomic writes, transaction failure handling
SDE-2 ready	reconciles unknown outcomes, plans operations, measures trade-offs

Selected authoritative references

- MySQL 8.4 Reference Manual: InnoDB architecture, indexes, transactions, locking, optimizer, replication, backup, and Performance Schema.
- MySQL Connector/J Developer Guide: JDBC URL, properties, batching, timeouts, and type mapping.
- Java SE 21 JDBC API: [DataSource](#), [Connection](#), [PreparedStatement](#), [ResultSet](#), and [SQLException](#) contracts.

Verify examples against the database and driver version used by your target company. H2 examples in the companion lab validate portable SQL/JDBC behavior; they do not prove MySQL collation, InnoDB plan, lock-range, replication, or DDL semantics.

SDE-2 CORE CHAPTER

Chapter 11 - Java 21 MySQL Interview Reasoning Companion

Executable models for SQL truth, composite-index prefixes, MVCC visibility, optimistic updates, keyset boundaries, and transaction retry classification.

JAVA (continued 1/4)

```
import java.sql.SQLException;
import java.time.Instant;
import java.util.Comparator;
import java.util.List;
import java.util.Objects;

/** Dependency-free executable models for the MySQL interview volume. */
public final class MySqlInterviewCompanion {
    private MySqlInterviewCompanion() {}

    enum SqlTruth {
        TRUE, FALSE, UNKNOWN;

        SqlTruth and(SqlTruth other) {
            Objects.requireNonNull(other, "other");
            if (this == FALSE || other == FALSE) {
                return FALSE;
            }
            return this == UNKNOWN || other == UNKNOWN ? UNKNOWN : TRUE;
        }
    }

    record CompositeIndex(List<String> columns) {
        CompositeIndex {
            columns = List.copyOf(columns);
            if (columns.isEmpty()) {
                throw new IllegalArgumentException("an index needs a column");
            }
        }
    }
}
```

JAVA (continued 2/4)

```

    boolean canSeekByEqualityPrefix(List<String> equalityColumns) {
        if (equalityColumns.size() > columns.size()) {
            return false;
        }
        return columns.subList(0, equalityColumns.size()).equals(equalityColumns);
    }
}

record RowVersion(String value, long committedAtSequence, boolean deleted) {}

static String visibleValue(List<RowVersion> newestFirst, long readSequence) {
    return newestFirst.stream()
        .filter(version -> version.committedAtSequence() <= readSequence)
        .findFirst()
        .filter(version -> !version.deleted())
        .map(RowVersion::value)
        .orElse(null);
}

record VersionedOrder(long id, String status, long version) {
    VersionedOrder updateIfVersion(String nextStatus, long expectedVersion) {
        Objects.requireNonNull(nextStatus, "nextStatus");
        return version == expectedVersion
            ? new VersionedOrder(id, nextStatus, version + 1)
            : this;
    }
}

```

JAVA (continued 3/4)

```

record Cursor(Instant createdAt, long id) {
    Cursor {
        Objects.requireNonNull(createdAt, "createdAt");
    }
}

static boolean comesAfterInDescendingFeed(Cursor candidate, Cursor boundary) {
    int timeComparison = candidate.createdAt().compareTo(boundary.createdAt());
    return timeComparison < 0 || (timeComparison == 0 && candidate.id() < boundary.id());
}

static final Comparator<Cursor> NEWEST_FIRST = Comparator
    .comparing(Cursor::createdAt, Comparator.reverseOrder())
    .thenComparing(Comparator.comparingLong(Cursor::id).reversed());

static boolean isRetryableTransactionFailure(SQLException failure) {
    Objects.requireNonNull(failure, "failure");
    // SQLState 40001 is serialization/deadlock class. MySQL error codes 1213
    // (deadlock) and 1205 (lock wait timeout) require policy-specific retries.
    return "40001".equals(failure.getSQLState())
        || failure.getErrorCode() == 1213
        || failure.getErrorCode() == 1205;
}

public static void main(String[] args) {
    assert SqlTruth.FALSE.and(SqlTruth.UNKNOWN) == SqlTruth.FALSE;
    assert SqlTruth.TRUE.and(SqlTruth.UNKNOWN) == SqlTruth.UNKNOWN;
}

```

JAVA (continued 4/4)

```
CompositeIndex index = new CompositeIndex(
    List.of("customer_id", "status", "created_at", "order_id"));
assert index.canSeekByEqualityPrefix(List.of("customer_id", "status"));
assert !index.canSeekByEqualityPrefix(List.of("status"));

List<RowVersion> versions = List.of(
    new RowVersion("PAID", 12, false),
    new RowVersion("CREATED", 8, false));
assert "CREATED".equals(visibleValue(versions, 10));
assert "PAID".equals(visibleValue(versions, 12));

VersionedOrder original = new VersionedOrder(7, "CREATED", 3);
assert original.updateIfVersion("PAID", 2).equals(original);
assert original.updateIfVersion("PAID", 3).version() == 4;

Instant sameTime = Instant.parse("2026-08-02T10:00:00Z");
Cursor boundary = new Cursor(sameTime, 50);
assert comesAfterInDescendingFeed(new Cursor(sameTime, 49), boundary);
assert !comesAfterInDescendingFeed(new Cursor(sameTime, 51), boundary);

SQLException deadlock = new SQLException("deadlock", "40001", 1213);
assert isRetryableTransactionFailure(deadlock);
assert !isRetryableTransactionFailure(new SQLException("syntax", "42000", 1064));

System.out.println("MySQLInterviewCompanion checks passed");
}
```


Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: schemas and queries
- Interview Core: indexes and transactions
- SDE-2 Follow-up: plans, locks, migrations, and pools

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

This is the first volume in the Frameworks, Data, and Messaging segment.

[Series index](#)

[Next book: FW 02 - Hibernate and JPA for Java Backend Interviews](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Frameworks, Data, and Messaging - Book 01 of 12 - Study Step FW-01. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.